Numerical Logical Program :

```
---------------------------------
/*Check a number is Palindrome or not.
 A number is a palindrome If it reads same forward and backward.
 Example : 121 <-> 121
*/

/*
Logic to Check a Palindrome Number:
----------------------------------
1) Store the original number in a temporary variable.
2) Initialize reverse = 0
3) Reverse the number:
   a) Extract the last digit using num % 10, till num!=0
   b) Add it to the reversed number: reverse = (reverse * 10) + digit.
     c) Remove the last digit from the original number: num = num / 10.
4) Compare the reversed number with the temp variable.
  If both are equal, the number is a palindrome; otherwise, it is not.
*/

 void main()
 {
      int number = Integer.parseInt(IO.readln("Enter a number :"));
      IO.println("Is "+number+" palindrome number :"+isPalindrome(number));
 }

 public boolean isPalindrome(int num)  //num = 121
 {
   int temp = num;
   int reverse = 0; //121

    while(num !=0)  //num = 0
      {
           int digit = num % 10;  //digit = 1
           reverse = (reverse * 10) + digit;
           num = num /10;
      }

      return temp == reverse;
 }
```

```
/* Check a number is Strong number or not.

[If the sum of factorials of all individual digits of a number is equal to the number itself, then it is called a Strong Number.]

Example 145
!1 + !4 + !5 = 145

Example :
40585
!4 + !0 + !5 + !8 + !5

*/

/*
Strong number logic :
----------------------
1) Store the original number in a temporary variable.
2) Initialize sum = 0;
3) Extract each digit using % 10 till num !=0
4) Find the factorial of the digit. [getFactorial()]
5) Add the factorial to a sum.
6) Remove the last digit using / 10.
After processing all digits, compare the sum with the temporary number.
If equal, it's a Strong Number.
*/


void main()
{
     int number = Integer.parseInt(IO.readln("Enter the number :"));
     IO.println("Is "+number+" Strong number :"+isStrong(number));
}

boolean isStrong(int num)  //145
{
  int temp = num;
  int sum = 0;

  while(num !=0) //num = 0
  {
       int digit = num % 10;  //digit = 4
       sum = sum + getFactorial(digit);
       num = num /10;
  }

  return temp == sum;
}

int getFactorial(int n)                    Note : We are reusing the getFactorial() method to find out the
{                                                 factorial of each individual digit
  int fact = 1;

  for(int i=1; i<=n; i++)
     {
       fact = fact * i;
     }

     return fact;
}
```

```
/*
Check a number is Spy Number or not.
A number is called a Spy Number if, Sum of digits = Product of digits
123 ,1124.

1+2+3 = 1*2*3

*/

 /*
 Spy number logic :
 ------------------
 1) Initialize sum = 0 and prod = 1.
 2) Extract each digit using % 10 till num!=0
 3) Add the digit to sum.
 4) Multiply the digit with product.
 4) Remove the last digit using / 10.
 5) Compare sum and product.
 If equal, it is a Spy Number.
 */

void main()
{
     int num = Integer.parseInt(IO.readln("Enter a number :"));
     IO.println("Is "+num+" spy number :"+isSpy(num));
}

public boolean isSpy(int num)
{
     int sum = 0;
     int prod = 1;

    while(num !=0)
      {
          int digit = num % 10;
          sum = sum + digit;
       prod = prod * digit;
          num = num /10;
      }

    return sum == prod;

}
```

```
/* Check a number is prime Number or not.

If a number is divisible by 1 and itself (exactly two factors) then It is
called prime number.

2, 3, 5, 7, 11 -> Prime
1, 4, 6, 9 ->  Not Prime
*/

/*
Prime number logic :
----------------------
1) If the number is less than or equal to 1, it is not prime.
2) Check if the number is divisible by any number from 2 to num/2.
3) If divisible, it is not prime.
4) If no divisors are found, it is prime.
*/

 void main()
 {
      int num = Integer.parseInt(IO.readln("Enter a number :"));
      IO.println("Is "+num+" prime number :"+isPrime(num));
 }

 /*public boolean isPrime(int num)
 {
   if(num <=1)
      {
        return false;
      }

    for(int i=2; i<=num/2; i++)
      {
          if(num % i==0)
          {
               return false;
          }
      }

    return true;
 }*/


 public boolean isPrime(int num)
 {
      int count = 0;
      for(int i=1; i<=num; i++)
      {
      if(num % i==0)
          {
            count++;
          }
      }

    return count == 2;

 }

Assignment :
---------------
WAP to find all the prime numbers from 1 to 100.
```

```
/* Check a number is duck Number or not.

A number is called a Duck Number if:
It contains at least one zero (0) and must be positive number
The number should NOT start with zero

1708
170
0299 -> Not a duck number
*/


/*
Logic for Duck numer :
----------------------
1) Take the number as input.
2) if number is zero OR negative return false
3) initialize hasZeroDigit = false;
4) Extract each digit using % 10 till num !=0
5) Check if any digit is 0.
6) If a 0 is found, make hasZeroDigit true, use break.
7) Remove the last digit using / 10.
8) return hasZeroDigit
*/


 void main()
 {
      int number = Integer.parseInt(IO.readln("Enter a number :"));
      IO.println("Is "+number+" duck number :"+isDuck(number));
 }

 boolean isDuck(int num)  //102
 {
   if(num ==0 || num < 0)
      {
       return false;
      }

      boolean hasZeroDigit = false;

      while(num !=0)
      {
          int digit = num % 10;

          if(digit==0)
          {
               IO.println("If condition is true");
               hasZeroDigit = true;
               break;
          }
          num = num /10;
      }

    return hasZeroDigit;

 }
```